

EasyXT

回测系统文档 · 从数据到绩效分析

作者	王者quant
版本	V1.0
依赖	DuckDB + easyxt_backtest
更新时间	2026 年 7 月

目录

一、回测系统概述

1.1 系统架构

1.2 核心组件

二、数据准备

2.1 DuckDB 数据下载

2.2 DataManager 初始化

2.3 数据源降级

三、编写策略

3.1 策略基类

3.2 MA 双均线策略示例

3.3 红利低波策略示例

四、运行回测

4.1 BacktestEngine 参数

4.2 回测配置详解

4.3 调仓与再平衡

五、绩效分析

5.1 核心指标

5.2 可视化输出

5.3 基准对比

六、101 因子平台

6.1 平台启动

6.2 因子分析流程

6.3 因子中性化

七、进阶主题

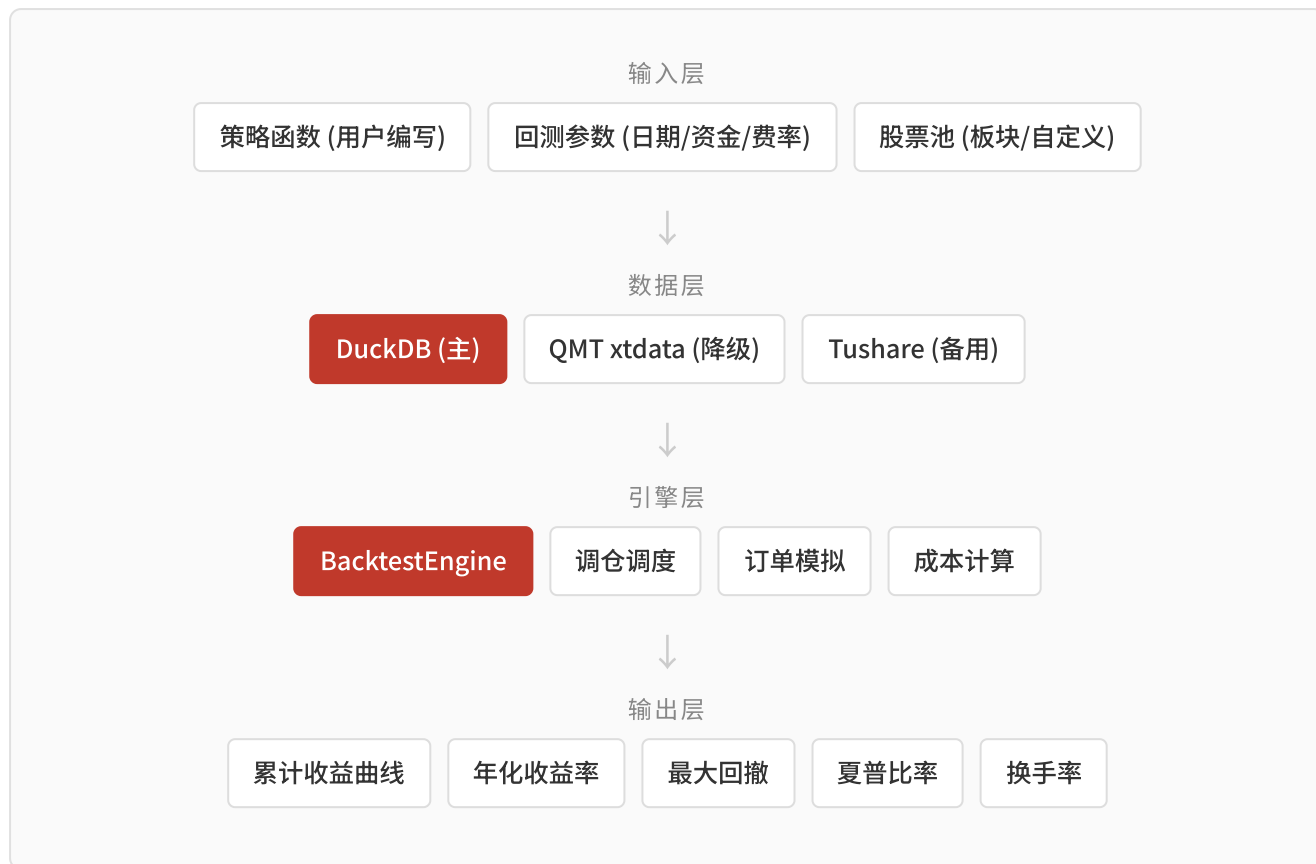
7.1 多因子模型

7.2 ML 选股集成

7.3 行业/市值中性化

一 回测系统概述

1.1 系统架构



1.2 核心组件

组件	模块	职责
DataManager	<code>easyxt_backtest.core</code>	数据加载、DuckDB 连接、股票池管理
BacktestEngine	<code>easyxt_backtest.core</code>	回测主循环、订单执行、绩效计算
Portfolio	<code>easyxt_backtest.portfolio</code>	组合构建、权重分配、再平衡
Factors	<code>easyxt_backtest.factors</code>	因子计算引擎、191 个 Alpha 因子
Filters	<code>easyxt_backtest.filters</code>	股票筛选（ST 排除、次新股、停牌等）
Scoring	<code>easyxt_backtest.scoring</code>	多因子打分、排名加权
Analysis	<code>easyxt_backtest.analysis</code>	绩效分析、可视化、基准对比
ML	<code>easyxt_backtest.ml</code>	机器学习选股（随机森林、XGBoost 等）

二 数据准备

2.1 DuckDB 数据下载

回测系统依赖 DuckDB 中的历史行情数据。数据下载方式：

1. **GUI 下载（推荐）**： `python run_gui.py` → "数据管理" → "下载A股数据"
2. **Tushare 下载**： GUI "Tushare下载" 标签页，选"日线行情"批量下载
3. **大QMT 导入**： 先在 QMT 客户端补充数据，再"从大QMT导入"

数据文件默认路径： `D:/StockData/stock_data.ddb`

数据覆盖建议：回测需要覆盖完整的回测区间。如果要回测 2020-2025 年，确保 DuckDB 中 `stock_daily` 表的日期范围包含 2019-01-01（需要至少 1 年预热数据用于计算均线等技术指标）。

2.2 DataManager 初始化

```
from easyxt_backtest import DataManager

# 自动检测 DuckDB 路径（D盘 → C盘 → E盘）
dm = DataManager()

# 手动指定路径
dm = DataManager(duckdb_path='D:/StockData/stock_data.ddb')

# 查看数据概况
print(f"股票数: {dm.stock_count}")
print(f"日期范围: {dm.date_range}")
print(f"数据记录: {dm.total_records:,}")
```

2.3 数据源降级

当 DuckDB 数据不足时，回测系统会自动降级：

优先级	数据源	速度	条件
1	DuckDB	毫秒级	已下载历史数据
2	QMT xtdata	秒级	QMT 已启动且本地有缓存
3	Tushare	分钟级	有 Token 且有积分

三 编写策略

3.1 策略函数接口

回测引擎要求策略是一个函数，签名为：

strategy_func(trade_date, data_manager, current_positions) → dict

参数	类型	说明
trade_date	str	当前调仓日期，格式 'YYYYMMDD'
data_manager	DataManager	数据管理器，提供 get_price() 等方法
current_positions	Dict[str, int]	当前持仓 {code: shares}

返回值： Dict[str, float] ，目标权重 {code: weight} ，权重之和应为 1.0。

返回空字典 {} 表示清仓（全部卖出）。

3.2 MA 双均线策略示例

经典的 MA5 上穿 MA20 买入，持有至死叉卖出：


```

def ma_cross_strategy(trade_date, data_manager, current_positions):
    """MA5 上穿 MA20 买入，下穿卖出"""
    import pandas as pd

    # 从股票池中选择
    stock_pool = ['000001.SZ', '600519.SH', '000858.SZ', '601318.SH', '000333.SZ']

    buy_list = []
    for code in stock_pool:
        # 获取最近 60 天数据（足够算均线）
        df = data_manager.get_price([code],
                                     start=(pd.Timestamp(trade_date) - pd.Timedelta(days=120)).strftime('%Y%m%d'),
                                     end=trade_date, period='1d', adjust='front_ratio')

        if len(df) < 20:
            continue

        close = df['close']
        ma5 = close.rolling(5).mean().iloc[-1]
        ma20 = close.rolling(20).mean().iloc[-1]
        prev_ma5 = close.rolling(5).mean().iloc[-2]
        prev_ma20 = close.rolling(20).mean().iloc[-2]

        # 金叉买入
        if prev_ma5 <= prev_ma20 and ma5 > ma20:
            buy_list.append(code)

    # 等权分配
    if buy_list:
        weight = 1.0 / len(buy_list)
        return {code: weight for code in buy_list}

    return {} # 无信号，清仓

```

3.3 红利低波策略示例

完整示例见项目内置策略 `strategies/quant_strategies/dividend_lowvol.py`，核心思路：DuckDB 读分红数据 → 计算股息率 → 60 日波动率筛选 → 排名打分 → 买入前 N。

```
def dividend_lowvol_strategy(trade_date, data_manager, current_positions):  
    # 1. 获取全市场股票  
    all_stocks = data_manager.get_stock_list()  
  
    # 2. 获取分红数据（从 DuckDB dividend_data 表）  
    div_data = data_manager.get_dividend_data()  
  
    # 3. 计算股息率 + 波动率，排名筛选  
    # ... (完整逻辑见 dividend_lowvol.py)  
  
    # 4. 返回目标权重  
    return {code: 1.0/top_n for code in selected}
```

四 运行回测

4.1 BacktestEngine 参数

BacktestEngine(initial_cash, data_manager, commission, slippage, benchmark)

参数	类型	默认值	说明
initial_cash	float	必填	初始资金（元），如 1000000
data_manager	DataManager	必填	数据管理器实例
commission	float	0.00025	手续费率，默认万 2.5
slippage	float	0.001	滑点，默认 0.1%
benchmark	str	'000300.SH'	基准指数代码

4.2 回测配置详解

```
from easyxt_backtest import DataManager, BacktestEngine

# 1. 准备数据
dm = DataManager()

# 2. 创建引擎
engine = BacktestEngine(
    initial_cash=1000000,      # 100 万初始资金
    data_manager=dm,
    commission=0.00025,       # 万 2.5 手续费
    slippage=0.001,           # 0.1% 滑点
    benchmark='000300.SH'     # 沪深 300 基准
)

# 3. 运行回测
result = engine.run_backtest(
    strategy=ma_cross_strategy,
    start_date='2024-01-01',
    end_date='2024-12-31',
    rebalance_freq='monthly',  # 月度调仓
    stock_pool=['000001.SZ', '600519.SH', '000858.SZ', '601318.SH', '000333.SZ']
)

# 4. 查看结果
result.print_summary()
result.plot_equity_curve()
```

4.3 调仓与再平衡

参数值	说明	适用场景
'daily'	每日调仓	高频策略、日内信号
'weekly'	每周调仓	中频轮动
'monthly'	每月调仓（推荐）	大多数因子策略
'quarterly'	每季调仓	低频价值策略

调仓频率与换手率的关系：更高频的调仓会增加换手率和交易成本。月度调仓是 A 股因子策略的最佳平衡点——既能及时捕捉因子 alpha，又不会产生过高交易成本。

五 绩效分析

5.1 核心指标

回测完成后，`result` 对象提供以下关键绩效指标：

指标	英文	计算公式	说明
累计收益率	Total Return	$(\text{最终净值} - 1) \times 100\%$	回测期间的总收益
年化收益率	CAGR	$(\text{最终净值})^{(252/\text{交易日数})} - 1$	年化复合增长率
年化波动率	Volatility	$\text{日收益标准差} \times \text{sqrt}(252)$	风险度量
夏普比率	Sharpe Ratio	$(\text{年化收益} - \text{无风险利率}) / \text{年化波动率}$	风险调整后收益，>1 为良好
最大回撤	Max Drawdown	$\text{max}(\text{峰值} - \text{谷值}) / \text{峰值}$	历史最大亏损幅度
胜率	Win Rate	$\text{盈利调仓次数} / \text{总调仓次数}$	每次调仓的盈利概率
盈亏比	Profit/Loss Ratio	$\text{平均盈利} / \text{平均亏损}$	大于 1 说明盈亏结构良好
换手率	Turnover	每次调仓的买卖金额占比	过高则交易成本侵蚀收益
信息比率	IR	$\text{超额收益} / \text{跟踪误差}$	相对基准的超额稳定性

5.2 可视化输出

```
# 打印摘要
result.print_summary()

# 绘制净值曲线（策略 vs 基准）
result.plot_equity_curve()

# 绘制月度收益热力图
result.plot_monthly_returns()

# 绘制回撤曲线
result.plot_drawdown()

# 导出详细报告
result.export_report('backtest_report.html')
```

5.3 基准对比

回测引擎自动计算相对基准的超额收益：

基准代码	指数名称	代表
'000300.SH'	沪深 300	大盘蓝筹
'000905.SH'	中证 500	中盘成长
'000852.SH'	中证 1000	小盘
'399006.SZ'	创业板指	创业板

六 101 因子平台

6.1 平台启动

101 因子分析平台是基于 Streamlit 的独立 Web 应用：

```
cd EasyXT/101因子/101因子分析平台
streamlit run main_app.py
```

或者从项目根目录：

```
streamlit run 101因子/101因子分析平台/main_app.py
```

启动后浏览器自动打开，界面分为以下模块：

模块	功能
 因子分析	选择因子 → IC 分析 → 分层回测 → 因子分布可视化
 信号生成	多因子组合 → 加权/排名融合 → 生成交易信号
 组合构建	等权/市值加权/因子加权 → 行业偏离约束
 回测验证	参数设置 → 运行回测 → 绩效报告

6.2 因子分析流程



↓

STEP 4: 信号 & 组合 & 回测

多因子信号

组合优化

绩效评估

6.3 因子中性化

在构建多因子模型时，某些因子可能被行业或市值风格所主导。例如"低市盈率"因子可能天然偏向银行行业。中性化处理可以剥离这些干扰：

中性化方式	方法	说明
行业中性化	因子值对行业哑变量回归，取残差	消除行业偏好
市值中性化	因子值对 log(市值) 回归，取残差	消除大小盘偏好
行业+市值双中性化	同时对两者回归取残差	最常用的配置

```
# 在 101 因子平台中操作：
# 信号生成 → 勾选 "行业中性化" + "市值中性化" → 生成信号
```


七 进阶主题

7.1 多因子模型

将多个因子组合成一个综合信号：

组合方式	方法	优点	缺点
等权组合	所有因子权重相同	简单、稳健	好因子权重不足
IC 加权	按历史 IC 值分配权重	给有效因子更高权重	可能过拟合
ICIR 加权	按 IC 信息比率分配权重	同时考虑有效性和稳定性	需要足够长的 IC 序列
排名融合	对每个因子的排名加权求和	量纲无关、极端值鲁棒	丢失原始因子值信息

7.2 ML 选股集成

`easyxt_backtest.ml` 模块提供了机器学习辅助选股能力：

```
from easyxt_backtest.ml import MLStrategy

# 使用随机森林分类器预测下期涨跌
ml = MLStrategy(
    model='random_forest',      # 模型：random_forest / xgboost / lgbm
    factors=['alpha001', 'alpha002', 'alpha006', 'alpha012'],
    lookback=60,                # 用过去 60 天数据训练
    retrain_freq='monthly'      # 每月重新训练
)

# 在回测中使用 ML 策略
result = engine.run_backtest(strategy=ml.predict, ...)
```

7.3 行业/市值中性化

为什么需要中性化？以市盈率因子为例：

场景	中性化前	中性化后
选股结果	90% 银行股 + 10% 其他	各行业均衡分布
行业集中度	极高（银行一家独大）	合理分散
回撤风险	银行板块下跌则策略崩盘	行业分散降低风险
因子纯度	PE 因子夹杂了行业偏好	PE 因子更纯

建议：大多数多因子模型应使用行业+市值双中性化。只有当你确信你的因子与行业/市值无关（如纯粹的量价因子），才无需中性化。

— 文档结束 —

EasyXT 量化交易框架 · 回测系统文档

王者quant · 2026 年 7 月